Rapid Web Development with Python/Django

Models

B. Wakim

Outline

- **Introduction to Django**
  - What is Django
  - Why Django for Web Development
- **Django Architecture**
  - MVC Design in Django
  - Steps to create New Project
- **Django Models**
  - Description
  - Field types
  - Field options
  - Relationships
  - Methods
  - Activating a model
- **Ressources**

Introduction to Django

Introduction

- Web applications are written using a combination of Python and JavaScript. It is executed on the server side while JavaScript is downloaded to the client and run by the web browser.
- Web applications created in Python are often made with the Flask or Django module.
- A web framework is a collection of packages and modules that allows easier development of websites.
- Most companies like Bitbucket, Pinterest, Instagram and Dropbox used to Python frameworks by Pyramid and Django in their **web application development**.

What is Django

- Django is a web development framework that developers primarily use for backend web development. It is written in Python and provides a set of tools and libraries to help you build web apps quickly and efficiently.
- **History**
  - Open sourced in 2005
  - First Version released September 3, 2008
- **Concepts**
  - MVC or MVT
  - Flexible template language that can be used to generate HTML, CSV, Email or any other format
  - Supports many databases – Postgresql, MySQL, Oracle, SQLite
  - Lots of extras included – middleware, sessions, caching, authentication
  - DRY Principle – "Don't Repeat Yourself". It is a principle of software development that **aims at reducing the repetition of patterns and code duplication in favor of abstractions and avoiding redundancy**.
  - Use small, reusable "apps" (app = python module with models, views, templates, test)

Why Django for Web Development

- Lets you divide code modules into logical groups to make it flexible to change: *MVC design pattern (MVT)*
- Provides auto generated web admin to ease the website administration
- Provides pre-packaged API for common user tasks
- Provides you template system to define HTML template for your web pages to avoid code duplication: *DRY Principle*
- Allows you to define what URL be for a given Function
- Allows you to separate business logic from the HTML: *Separation of concerns*
- Everything is in python

Django Architecture

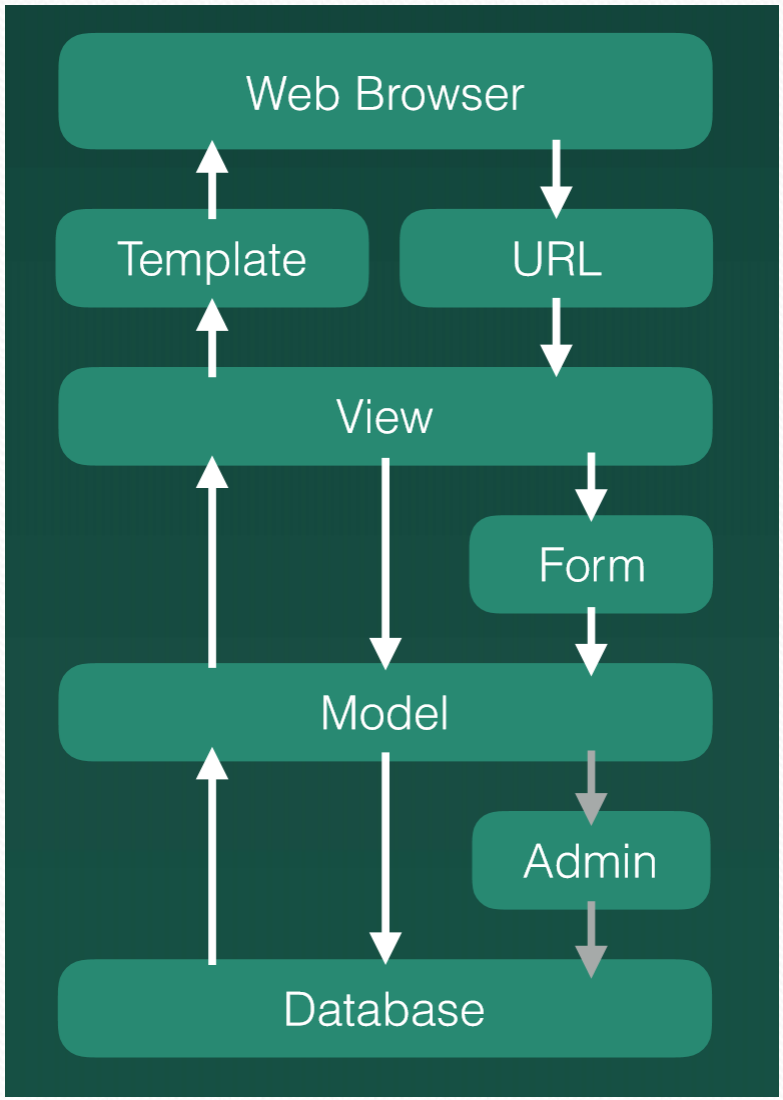
Django as an MVC Design Pattern

- **MVT** : stands for *Model-View-Template*. MVT is a design pattern or design architecture that Django follows to develop web applications. It is slightly different from the commonly known MVC(Model-View-Controller) design pattern.
- **Model:** helps us handle database: it describes your data structure/database schema
  - The Model class holds essential fields and methods. For each model class, we have a table in the database. Model is a subclass of django.db.models.Model.
  - With Django, we have a database-abstraction API that lets us perform CRUD (Create-Retrieve-Update-Delete) operations on mapped tables.
- **View:** Controls what a user sees
  - View executes business logic and interacts with Model to carry data. It also renders Template.

Django as an MVC Design Pattern

- **Template:** handles the user interface and is a presentation layer.
  - The Template is basically the front-end layer and the dynamic HTML component of a Django application.
  - A template is an HTML file mixed with DTL *(Django Template Language)*.
- **Controller**
  - *Django framework* takes care of the Controller part, which is the software code and controls the interaction between the other two parts- Model and View.
  - *URL parsing:* When a user requests for a resource, Django acts as a controller and checks if it is available. If the URL maps, View interacts with the Model and renders a Template. Python Django sends back a Template to the user as a response.

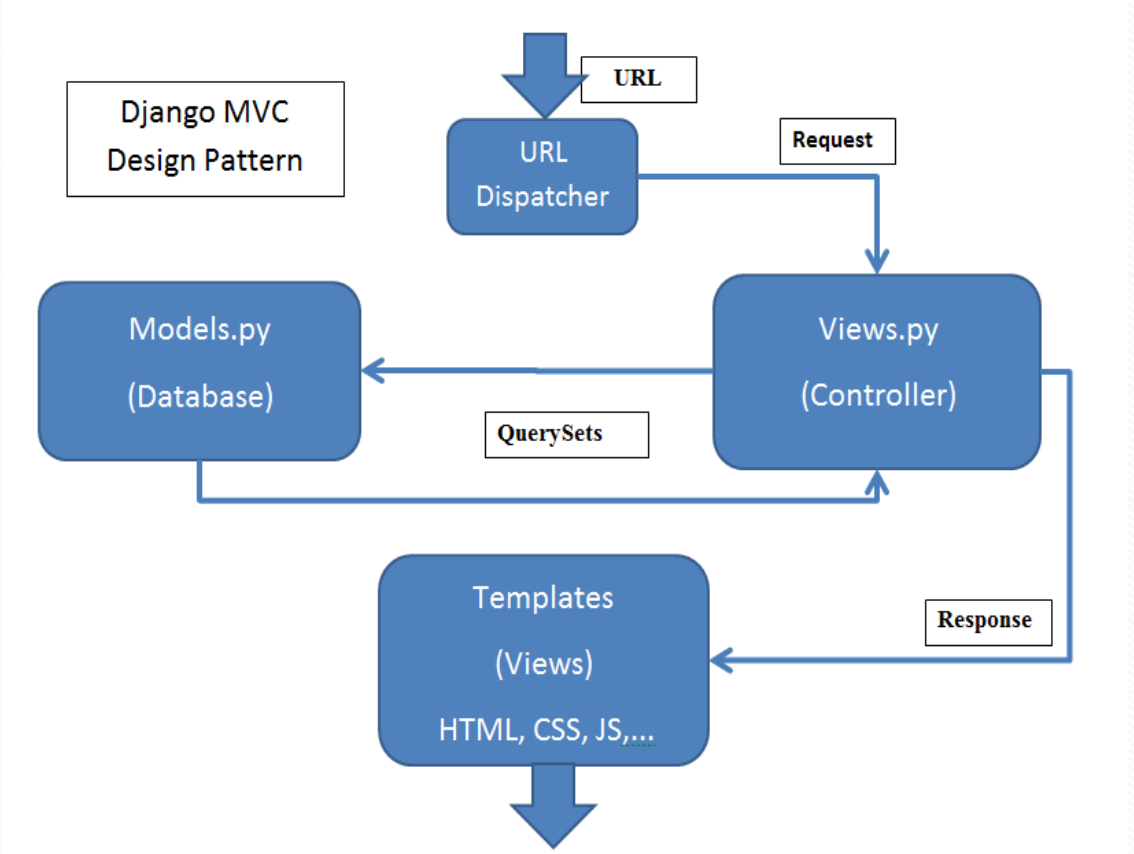
Django MVT



- **Web Browser**: What users actually see
- **URL**: the 'web address'
  - Provides mapping to View
  - Assigning functions to URLs
- **View**: Where all the functions are written
  - Render content to Template
  - Get information from Model before rendering content
  - Put information into Model and into Database through Form
- **Template**: where you store all of your Html files
  - You will have a 'static' folder to store other CSS files, JavaScript files, or Images
- **Form**:
  - HTML Form
  - Django Form: fetch data from html form to connect to Model
- **Model**: DB structure + access
- **Admin**: to register your object in your Model so you can manage data in Database

How things in Django works?

- When your server receives a request for a website, the request is passed to Django and that tries to analyze this request.
- The url resolver/dispatcher tries to match the URL against a list of patterns. It performs this match from top to bottom. If it can find a match, it passes the request to the view, which is the associated function.
- The function view can check if the request is allowed. It also generates a response, and then Django sends it to the user's web browser.



Steps to create New Project

- Create a project
- Start an application
- Create the database (MySQL, PostgreSQL, SQLite)
- Define DB Settings in **Settings.py**
- Define your models
- Add pluggable modules
- Write your templates
- Define your views
- Create **URL** mapping
- Test Application
- Deploy Application (Linux, Apache, etc.)

Directory Architecture

- MySite/
  - __init__.py
  - Manage.py // Script to interact with Django
  - Settings.py // Config: Defines settings used by a Django application
  - URLs.py$ // My Site URL mapping
  - MyProject/
    - __init__.py
    - URLs.py // Project specific URL mapping to provide mapping to view.py
    - Models.py // Data Models
    - Views.py // Contains the call back functions
    - Admin.py // It reads your model and provides interface to your database
    - Templates

Django Models

Django Models

- Defined in **models.py**
- Typically inherit from **django.db.models.Model**
- Example Model:

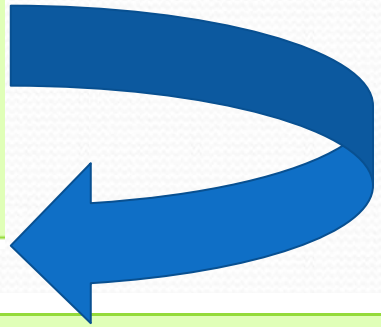
```
from django.db import models
class TestModel(models.Model):
    # Default is to set NOT NULL on all fields.
    name = models.CharField(max_length = 20, null = True)
    age = models.IntegerField()
    count = models.IntegerField(default = 0)
```

Models description

- Is the single, definitive source of data about your data.
- Contains the essential fields and behaviours of the data you are storing.
- Each model maps to a single database table.
- **Example**: Person model would create a database table like this:

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```



```
CREATE TABLE myapp_person (
    "id" serial NOT NULL PRIMARY KEY,
    "first_name" varchar(30) NOT NULL,
    "last_name" varchar(30) NOT NULL
);
```

Models

Example:

```
class Musician(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    instrument = models.CharField(max_length=100)

class Album(models.Model):
    artist = models.ForeignKey(Musician)
    name = models.CharField(max_length=100)
    release_date = models.DateField()
    num_stars = models.IntegerField()
```

- **Field types**
  - The database column type (e.g. INTEGER, VARCHAR).
  - The minimal validation requirements, used in Django's admin and in automatically-generated forms.

Models - field types

Data type	Django model type
Binary	models.BinaryField()
Boolean	models.BooleanField()
Boolean	models.NullBooleanField()
Date/time	models.DateField()
Date/time	models.TimeField()
Date/time	models.DateTimeField()
Date/time	models.DurationField()
Number	models.AutoField()
Number	models.BigIntegerField()
Number	models.DecimalField(decimal_places=X,max_digits=Y)
Number	models.FloatField()
Number	models.IntegerField()
Number	models.PositiveIntegerField()
Number	models.PositiveSmallIntegerField()
Number	options.SmallIntegerField()
Text	models.CharField(max_length=N)
Text	models.TextField()
Text (Specialized)	models.CommaSeparatedIntegerField(max_length=50)
Text (Specialized)	models.EmailField()
Text (Specialized)	models.FileField()
Text (Specialized)	models.FilePathField()
Text (Specialized)	models.ImageField()
Text (Specialized)	models.GenericIPAddressField()
Text (Specialized)	models.URLField()

Models - Field options

Field options	Description
null	If True, Django will store empty values as NULL in the database. Default is False.
blank	If True, the field is allowed to be blank. Default is False. null is purely database-related, whereas blank is validation-related. If a field has blank=True, form validation will allow entry of an empty value
choices	A sequence consisting itself of iterables of exactly two items (e.g. [(A, B), (A, B) ...]) to use as choices for this field.
db_column	The name of the database column to use for this field. If this isn't given, Django will use the field's name.
db_index	If True, a database index will be created for this field.
db_tablespace	The name of the database tablespace to use for this field's index, if this field is indexed.
default	The default value for the field. This can be a value or a callable object.
editable	f False, the field will not be displayed in the admin or any other ModelForm. They are also skipped during model validation
primary_key	If True, this field is the primary key for the model.
unique	If True, this field must be unique throughout the table.
unique_for_date	et this to the name of a DateField or DateTimeField to require that this field be unique for the value of the date field.
unique_for_month	Like unique_for_date, but requires the field to be unique with respect to the month.
unique_for_year	Like unique_for_date and unique_for_month.
verbose_name	human-readable name for the field. If the verbose name isn't given, Django will automatically create it using the field's attribute name, converting underscores to spaces.
help_text	Extra "help" text to be displayed with the form widget. It's useful for documentation even if your field isn't used on a form.

Models – Field options

- **Field options:** Each field takes a certain set of field-specific arguments.
- For example, **CharField** arguments:
  - require a max_length argument which specifies the size of the VARCHAR database field used to store the data.
  - null, blank, choices, default, primary_key, unique
- Default is to set NOT NULL on all fields. Override by adding null = True :

  name = models.CharField(max_length=20, null = True)
- **Verbose field names:** are optional. They are used if you want to make your model attribute more readable. If not defined, django automatically creates it using field's attribute name.

In this example, the verbose name is "person's first name":

```
first_name = models.CharField("person's first name", max_length=30)
```

In this example, the verbose name is "first name":

```
first_name = models.CharField(max_length=30)
```

Models – Field options

- If **choices** are given, they're enforced by model validation and the default form widget will be a select box with these choices instead of the standard text field.
- The first element in each tuple is the actual value to be set on the model, and the second element is the human-readable name. For example:

```
YEAR_IN_SCHOOL_CHOICES = [
    ('FR', 'Freshman'),
    ('SO', 'Sophomore'),
    ('JR', 'Junior'),
    ('SR', 'Senior'),
    ('GR', 'Graduate'),
]
```

Models – Field options

- It's best to define **choices** inside a model class, and to define a constant for each value:

```
from django.db import models
class Student(models.Model):
    FRESHMAN = 'FR'
    SOPHOMORE = 'SO'
    JUNIOR = 'JR'
YEAR_IN_SCHOOL_CHOICES = [
        (FRESHMAN, 'Freshman'),
        (SOPHOMORE, 'Sophomore'),
        (JUNIOR, 'Junior'),
        (SENIOR, 'Senior'),
]
    year_in_school = models.CharField(max_length=2,
choices=YEAR_IN_SCHOOL_CHOICES,
        default=FRESHMAN )

    def is_upperclass(self):
        return self.year_in_school in {self.JUNIOR, self.SENIOR}
```

Models Relationship

- **Many-to-one:** use ForeignKey
  - requires a positional argument: the class to which the model is related.
- **Many-to-many:** use ManyToManyField.
  - requires a positional argument: the class to which the model is related.
- **One to one**: use OneToOneField
  - primary key of an object when that object "extends" another object.
  - requires a positional argument: the class to which the model is related.

```
class Manufacturer(models.Model):
    # ...

class Car(models.Model):
    manufacturer = models.ForeignKey(Manufacturer)
    # ...
```

```
class Topping(models.Model):
    # ...

class Pizza(models.Model):
    # ...
    toppings = models.ManyToManyField(Topping)
```

```
class Restaurant(models.Model):
    place = models.OneToOneField(Place, primary_key=True)
    serves_hot_dogs = models.BooleanField()
    serves_pizza = models.BooleanField()
```

Models Relationship

- Can have recursive relationship
- **N.B**: __Unicode has been renamed __str in Python3

```
class Person(models.Model):
    name = models.CharField(max_length=128)

    def __unicode__(self):
        return self.name

class Group(models.Model):
    name = models.CharField(max_length=128)
    members = models.ManyToManyField(Person, through='Membership')

    def __unicode__(self):
        return self.name

class Membership(models.Model):
    person = models.ForeignKey(Person)
    group = models.ForeignKey(Group)
    date_joined = models.DateField()
    invite_reason = models.CharField(max_length=64)
```

Model methods

```
from django.db import models
class Person(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    birth_date = models.DateField()

    def baby_boomer_status(self):
        "Returns the person's baby-boomer status."
        import datetime
        if self.birth_date < datetime.date(1945, 8, 1):
            return "Pre-boomer"
        elif self.birth_date < datetime.date(1965, 1, 1):
            return "Baby boomer"
        else:
            return "Post-boomer"

    @property
    def full_name(self):
        "Returns the person's full name."
        return '%s %s' % (self.first_name, self.last_name)
```

Model Methods

- model.save(self, *args, **kwargs)
- model.delete(self, *args, **kwargs)
- model.get_absolute_url(self): This tells Django how to calculate the URL for an object. Django uses this in its admin interface, and any time it needs to figure out a URL for an object.
- model.__str__(self)
- Override with super(MODEL, self).save(*args, **kwargs)
- Etc…

Model method - str

- **model.__str__(self):**
  - A Python “magic method” that returns a string representation of any object. Always define this method; the default isn’t very helpful at all.

```
Class MyclubUser(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    email = models.EmailField('User Email')

    def __str__(self):
        return self.first_name + " " + self.last_name
```

Model methods

- Override predefined method:
  - call the superclass method -- that's that **super(Blog, self).save(*args, **kwargs)** business -- to ensure that the object still gets saved into the database.
  - pass through the arguments that can be passed to the model method -- that's what the ***args, ****kwargs** bit does.

```
class Blog(models.Model):
    name = models.CharField(max_length=100)
    tagline = models.TextField()

    def save(self, *args, **kwargs):
        if self.name == "Yoko Ono's blog":
            return # Yoko shall never have her own blog!
        else:
            super(Blog, self).save(*args, **kwargs) # Call the "real" save() method.
```

Activating a Model

1. Add the app to INSTALLED_APPS in settings.py
2. python manage.py makemigrations myapp
3. python manage.py migrate
4. python manage.py check myapp
   # Uses the system check framework to inspect the app for common problems.

Database setup – settings.py¶

- **mysite/settings.py:** is a normal Python module with module-level variables representing Django settings.
- By default, the configuration uses SQLite.
- If you wish to use another database, install the appropriate database bindings and change the following keys in the DATABASES 'default' item to match your database connection settings:
  - ENGINE: 'django.db.backends.mysql'
  - NAME – The name of your database. If you are not using SQLite as your database, additional settings such as USER, PASSWORD, and HOST must be added.

Database setup – settings.py ¶

- Also, note the INSTALLED_APPS setting at the top of the file Settings.py. That holds the names of all Django applications that are activated in this Django instance.
- Some of these applications make use of at least one database table, though, so we need to create the tables in the database before we can use them. To do that, run the following command:

  python manage.py migrate
- The migrate command looks at the INSTALLED_APPS setting and creates any necessary database tables according to the database settings in your mysite/settings.py file and the database migrations shipped with the app

Database setup – settings.py

- To include your app in our project, we need to add a reference to its configuration class in the INSTALLED_APPS setting. The MyAppConfig class is in the polls/apps.py file, so its dotted path is ' Myapp.apps.MyAppConfig '.
- Edit the mysite/settings.py file and add that dotted path to the INSTALLED_APPS setting. It'll look like this:

```
mysite/settings.py¶
INSTALLED_APPS = [
    'Myapp.apps.MyAppConfig', - OR simply Myapp
    'django.contrib.admin', – The admin site.
    'django.contrib.auth', – An authentication system
    'django.contrib.contenttypes', – A framework for content types.
    'django.contrib.sessions', – A session framework.
    'django.contrib.messages', – A messaging framework.
    'django.contrib.staticfiles', – A framework for managing static files.
]
```

- N.B: This means that if the default_app_config in your myApp/__init__.py is already equal to myApp.apps.MyappConfig, then there is no difference between adding either myApp.apps.MyappConfig or simply myApp to the INSTALLED_APPS setting.

Database setup – migration

- By running **makemigrations**, you're telling Django that you've made some changes to your models (in this case, you've made new ones) and that you'd like the changes to be stored as a migration.

  python manage.py makemigrations Myapp

- The **migrate** command takes all the migrations that haven't been applied and runs them against your database - essentially, synchronizing the changes you made to your models with the schema in the database.

  python manage.py migrate

Ressources

- Django – http://www.djangoproject.com
- Python Packages – https://pypi.python.org
- Django Packages – https://www.djangopackages.com
- The best place to start

  http://docs.djangoproject.com/en/dev/
- **To install Django:**
- python -m pip install Django OR pip install django
- Check django version: Python –m django version
  - N.B: pip is the package installer for Python. You can use pip to install packages from the Python Package Index and other indexes.
- Django Models syntax: https://docs.djangoproject.com/en/3.2/topics/db/models/
- Django Models Fields Type https://docs.djangoproject.com/en/3.2/ref/models/fields/